

Montagem de /proc em um runtime Docker-like

Visão correta do PID namespace, procfs interno e suporte consistente a ps e top em uma arquitetura com control plane em Python, integração em Cython e data plane em C++.

Escopo. Construção de uma feature de runtime para um sistema semelhante ao Docker, feito do zero, em que a API, o estado desejado e os metadados residem em Python, enquanto a execução de baixo nível e a conversa com o kernel ficam em C++.

Data. 12 de março de 2026

Tese central. Um /proc correto não surge por bind mount do host nem por entrar “mais ou menos” em namespaces. Ele exige um procfs novo, montado no mount namespace do contêiner por um processo que já esteja no PID namespace correto. É isso que materializa a visão que ferramentas como ps e top esperam encontrar.

Sumário

Documento técnico sobre a implementação da feature de montagem de /proc em um runtime Docker-like escrito em Python, Cython e C++.

1. Resumo executivo
2. Conceitos fundamentais e definição do problema
3. Como o Docker e a pilha OCI tratam o tema
4. Arquitetura recomendada para Python, Cython e C++
5. System Design da feature
6. Low Level Design
7. Modelagem por Domain-Driven Design
8. Padrões de projeto aplicáveis
9. Requisitos funcionais e não funcionais
10. Plano de implementação por fases
11. Estratégia de testes e validação
12. Conclusão
13. Referências

Resumo executivo

A feature pedida não é “montar um diretório chamado `/proc`”. Ela é, na prática, a construção de uma visão processual coerente para o contêiner. Em Linux, essa coerência nasce da interação entre *PID namespace*, *mount namespace* e o ato de montar um novo *procfs* a partir do processo que já está no namespace correto. O detalhe crítico, frequentemente ignorado, é que o `/proc` só mostra os processos visíveis para o PID namespace do processo que realizou a montagem; por isso, entrar no PID namespace sem recriar o `/proc`, ou simplesmente bind-montar o `/proc` do host, produz uma visão errada para ferramentas como `ps` e, em muitos casos, para `top` [1] [2] [4] [7] [8].

Quando se observa como o Docker contemporâneo resolve esse problema, a resposta não aparece no daemon em si, mas na borda do runtime. O Docker Engine delega a execução a componentes como *containerd*, *shim* e *runc*; é o runtime OCI que cria namespaces, monta o *procfs*, executa o `pivot_root`, sobe o processo inicial e aplica endurecimentos como *masked-Paths* e *readonlyPaths* [12] [16] [17] [18]. Essa separação é especialmente útil para o projeto proposto, porque confirma uma diretriz arquitetural importante: a lógica semântica de alto nível, o estado desejado e os metadados ficam no control plane em Python, enquanto a montagem real do ambiente de execução — com syscalls, descritores de arquivo, sincronização entre pai e filho, regras de montagem e limpeza — pertence ao data plane em C++.

A recomendação central deste documento é adotar um modelo em três camadas. A primeira camada, em Python, concentra API, persistência, reconciliação e governança de políticas. A segunda, em Cython, atua como fronteira tipada e camada anticorrupção, convertendo DTOs, normalizando erros e reduzindo a fricção entre objetos Python e estruturas C++. A terceira, em C++, é um runtime dedicado, preferencialmente em processo separado do interpretador Python, responsável por `clone3`, `setns`, `mount`, `pivot_root`, `pidfd`, `waitid` e demais primitivas de kernel [5] [9] [10].

Decisão arquitetural recomendada

O perfil padrão deve ser um **procfs interno, novo e endurecido**, montado dentro do *mount namespace* do contêiner por um processo que já esteja no *PID namespace* alvo. Esse perfil deve usar, como base, as flags `MS_NOSUID`, `MS_NODEV` e `MS_NOEXEC`, reservar toda a subárvore `/proc` para o runtime e aplicar políticas de mascaramento e somente leitura nos caminhos sensíveis. O modo `subset=pid` deve existir apenas como perfil opcional, porque ele pode ser suficiente para enumeração de tarefas, mas tende a degradar utilitários que esperam arquivos globais como `/proc/meminfo`, `/proc/stat` ou `/proc/loadavg` [1] [8].

Também é recomendável tratar o caminho de `exec` como parte inseparável da feature. O erro clássico é assumir que um helper pode fazer `setns` apenas no PID namespace e então execu-

tar um shell. Isso não funciona como se imagina, porque a mudança de PID namespace via `setns()` afeta somente filhos futuros do chamador, não o chamador em si; além disso, se o helper continuar no mount namespace do host, continuará usando o /proc do host [5]. O desenho correto é: o helper entra primeiro no *mount namespace* do contêiner, depois reassocia o PID namespace para filhos, faz `fork` e executa o novo processo dentro da mesma visão de /proc já montada pelo runtime.

No plano de implementação, o caminho mais seguro é incremental. Primeiro vem um MVP rootful com PID namespace privado, mount namespace privado, novo procfs interno e verificação de ps. Depois entram init interno, *docker exec-like*, *pidfds*, endurecimento da subárvore /proc e proteção contra corridas de caminho. Em seguida chegam rootless e user namespaces, seguidos por recuperação após crash, coleta de lixo de mounts e testes de regressão inspirados nas vulnerabilidades históricas do *runc* [6] [10] [19] [20]. Em outras palavras: a feature deve ser tratada como uma capacidade de runtime de primeira classe, não como um detalhe cosmético de filesystem.

1. Conceitos fundamentais e definição do problema

1.1 O que é procfs e por que ele importa para contêineres

O *procfs* é um pseudo-filesystem exportado pelo kernel Linux, tradicionalmente montado em */proc*, que expõe estruturas de estado do sistema e do conjunto de processos em forma de arquivos e diretórios [1] [11]. Ao contrário de um filesystem persistente, ele não representa dados gravados em disco; ele é uma interface dinâmica entre o espaço de usuário e o estado interno do kernel. Isso tem duas consequências arquiteturais imediatas para um runtime de contêineres. A primeira é que */proc* não deve ser pensado como um artefato estático do root filesystem da imagem. A segunda é que qualquer tentativa de “simular” */proc* em espaço de usuário, seja via árvore de arquivos pré-populada, seja via FUSE artesanal, tende a ser semanticamente incompleta e operacionalmente frágil, porque ferramentas, bibliotecas e programas do ecossistema Linux assumem as particularidades reais do *procfs* do kernel.

No contexto do projeto, a expressão **procfs interno** deve receber uma definição precisa. Ela não significa bind-montar o */proc* do host dentro do rootfs do contêiner, nem significa implementar um filesystem sintético em C++ ou Python. Ela significa montar um *novo* *procfs*, controlado pelo runtime, no caminho */proc* do mount namespace do contêiner, com políticas de endurecimento próprias e acoplado semanticamente ao PID namespace correto. Essa distinção é central, porque a qualidade da feature depende da fidelidade semântica do *procfs*, não apenas da presença nominal do diretório.

Um erro recorrente em implementações caseiras de “container-like” é copiar um rootfs que já contém um diretório vazio chamado */proc* e considerar o problema resolvido. Não está. O diretório existe, mas as entradas mágicas, os symlinks especiais, os arquivos de status, as visões de namespace e os diretórios por PID só existem quando um *procfs* real foi montado pelo kernel [1] [11]. Esse é o primeiro princípio de projeto: */proc* não é dado da imagem; é infraestrutura de runtime.

1.2 PID namespaces e a semântica de visibilidade de processos

O PID namespace isola a numeração de processos e organiza essas visões em uma hierarquia. Um mesmo processo pode ter um PID diferente em cada namespace ancestral no qual é visível. O primeiro processo criado em um novo PID namespace torna-se o PID 1 daquele namespace, assumindo um papel análogo ao *init*: ele recebe responsabilidades especiais de reaping, tratamento de sinais e encerramento ordenado [2]. O manual do kernel é claro ao afirmar que, se o processo PID 1 do namespace terminar, o kernel encerra os processos remanescentes e impede novas criações de processos naquele namespace com falhas subsequentes de alocação [2]. Para um runtime, isso muda o desenho inteiro: sem um PID 1 interno confiável, não existe semântica robusta de contêiner.

Outro aspecto fundamental é a regra de visibilidade. Processos em um PID namespace filho podem ver processos daquele namespace e, conforme o caso, ser observados por namespaces ancestrais, mas o contrário não é simétrico; entrar em um PID namespace também é uma

operação de mão única em direção a descendentes, e não é permitido “subir” para namespaces ancestrais [2] [5]. Essa regra influencia diretamente a operação de *exec*, depuração e coleta de métricas.

A nuance decisiva para a feature é a seguinte: o `procfs` mostra apenas os processos visíveis no PID namespace do processo que efetuou a montagem [2]. Isso significa que a correção de `/proc` depende do instante e da identidade do montador. Se o runtime monta `procfs` antes de entrar no PID namespace correto, ou se herda um `procfs` já montado em outro contexto, a árvore `/proc` continuará descrevendo o universo errado. O PID namespace e o `procfs` estão relacionados, mas não são a mesma feature.

1.3 Mount namespaces e o fato de que a visão correta precisa viver em uma árvore de mounts própria

O mount namespace isola a tabela de montagens percebida pelo processo. Quando um novo mount namespace é criado, ele começa como uma cópia da lista de mounts do pai; a partir daí, mudanças podem divergir [4]. Essa característica é necessária, mas por si só não basta. Se o processo-filho herdar a árvore de mounts do host, inclusive um `/proc` já montado do lado de fora, o novo mount namespace conterá uma visão de `/proc` semanticamente ligada ao contexto anterior. Logo, criar somente um novo mount namespace sem substituir `/proc` não resolve o problema.

Além disso, um runtime sério precisa tornar a árvore de mounts privada logo no início do fluxo, geralmente com `mount(NULL, "/", NULL, MS_REC | MS_PRIVATE, NULL)`, para impedir propagação indesejada entre host e contêiner [4]. Sem essa etapa, alterações posteriores de mount podem vazar para fora ou ser influenciadas por propagação compartilhada. Em outras palavras, o mount namespace fornece o palco em que o `procfs` interno será materializado; se o palco não estiver isolado, a feature começa torta.

Essa observação ajuda a explicar casos em que um usuário faz `nsenter --pid`, executa `ps` e obtém resultados surpreendentes. Há relatos práticos exatamente nessa direção: entrar em um PID namespace sem alinhar o ambiente de mounts e o `procfs` pode preservar uma visão inconsistente para ferramentas de processos [21]. O problema não é o `ps`; o problema é a composição incompleta dos namespaces.

1.4 User namespaces, rootless e o modelo de capacidades

O user namespace é o mecanismo que permite separar identidade e privilégios entre host e contêiner. Dentro dele, um processo pode obter um conjunto de capacidades que não possui no namespace pai, desde que os mapas de UID e GID sejam estabelecidos corretamente [6] [13] [14]. No tema específico de `/proc`, isso importa porque a montagem de `procfs`, `sysfs`, `devpts`, `tmpfs` e outros filesystems especiais em ambientes rootless depende justamente da relação entre user namespace, mount namespace e, em alguns casos, PID namespace [6].

Para o projeto, isso leva a uma recomendação de desenho: o runtime precisa distinguir, como backends de baixo nível, pelo menos dois modos de operação. O primeiro é *rootful*, em que o processo do runtime possui as capacidades necessárias diretamente no host. O segundo é *rootless*, em que o runtime cria user namespace e usa a semântica de capacidades internas

para conseguir montar o `procfs` no ambiente do contêiner [6] [14]. A modelagem de domínio deve refletir essa distinção, porque os fluxos de sincronização, mapeamento de IDs e tratamento de erros não são os mesmos.

Há ainda uma implicação de ordem operacional. Quando `user namespace` e outros namespaces são criados juntos, o kernel estabelece o `user namespace` primeiro, o que é precisamente o que viabiliza o restante do setup em cenários `rootless` [6]. Um design que trate `user namespace` apenas como detalhe posterior tende a falhar em compatibilidade.

1.5 Por que `ps` e `top` são o oráculo prático da feature

O manual do `ps` deixa explícito que a ferramenta trabalha lendo arquivos virtuais em `/proc` [7]. Isso a transforma em um excelente oráculo de correção: se o contêiner tem um `/proc` semântica e topologicamente correto, o `ps` mostrará somente os processos visíveis naquele namespace, com PID 1 interno e relações de PPID coerentes. Se a montagem estiver errada, o defeito aparece imediatamente.

O `top` é um caso um pouco mais exigente. Embora ele também pertença ao ecossistema de ferramentas que vivem de `procfs` [8] [22], sua visão não se limita à enumeração de diretórios `/proc/<pid>`; na prática, ele costuma depender de arquivos globais como `/proc/meminfo`, `/proc/stat` e `/proc/loadavg` para compor sumários de CPU, memória e carga. Por isso, um perfil minimalista com `subset=pid`, disponível desde Linux 5.8, pode ser suficiente para cenários que exigem apenas visibilidade de tarefas, mas tende a ser insuficiente para uma experiência completa de observabilidade dentro do contêiner [1].

Essa diferença leva a uma decisão arquitetural simples e forte. Se a meta é paridade operacional com o ecossistema comum de Linux em contêineres, o perfil padrão não deve ser `subset=pid`; ele deve ser um `procfs` completo, endurecido e isolado. O perfil minimalista faz sentido como modo alternativo, não como base universal.

Estratégia	O que ela realmente faz	ps	top	Leitura arquitetural
Bind-mount do /proc do host	Expõe ao contêiner uma montagem produzida em outro contexto de namespace e de política de segurança.	Tende a mostrar a visão do host ou uma visão incoerente.	Tende a herdar sumários e arquivos globais do host.	É semanticamente incoerente para isolamento e arquivagem. Superfície de exposição não deve ser usada por default; idealmente, deve ser proibido.
Procfs novo com subset=pid	Monta um procfs real, mas pede ao kernel para esconder arquivos de nível superior não ligados a tarefas.	Em geral funciona para listagem de processos do namespace.	Pode ficar parcial ou perder informações esperadas por ferramentas de sumário.	É um bom perfil minimalista para workloads que não precisam de observabilidade completa dentro do contêiner.
Procfs novo completo e endurecido	Monta um procfs real no namespace correto, preserva arquivos globais e aplica máscara/somente leitura onde necessário.	Funciona com a visão correta do contêiner.	É o perfil mais compatível com o comportamento esperado de Linux em contêineres.	Deve ser o perfil padrão para qualquer projeto. Entrega completa sem abrir mão do endurecimento do /proc.

Erro conceitual frequente

É comum tratar a criação do PID namespace como se ela automaticamente corrigisse /proc. Não corrige. O PID namespace define a topologia de identificação e visibilidade de processos; o procfs montado no lugar certo materializa essa topologia para o espaço de usuário. Um runtime correto precisa das duas coisas, na ordem certa.

2. Como o Docker e a pilha OCI tratam o tema

Falar em “contextualizar no Docker original” exige separar ergonomia de produto e mecânica real de runtime. Do ponto de vista do usuário, o Docker popularizou a ideia de empacotar processos com isolamento, filesystem próprio, networking e uma experiência operacional uniforme. Do ponto de vista técnico contemporâneo, entretanto, o Docker Engine atua como camada de controle: o daemon gerencia imagens, rede, volumes, metadados e ciclo de vida de alto nível, mas a execução efetiva dos processos contidos é delegada à pilha formada por *containerd*, *shim* e runtime OCI, tipicamente *runc* [16] [17] [18]. É nessa borda do runtime que /proc deixa de ser uma abstração e vira syscall, mount flags e ordem de operação.

A documentação do Docker deixa claro que o padrão é rodar contêineres com PID namespace isolado, embora existam modos explícitos para compartilhar o PID namespace do host ou de outro contêiner, por exemplo via `--pid=host` ou `--pid=container:<id>` [12]. Esse detalhe é importante para o projeto porque mostra duas coisas ao mesmo tempo. Primeiro, isolamento de PID é o caso normal, não a exceção. Segundo, quando o usuário pede compartilhamento, isso precisa ser uma escolha deliberada de política, não uma consequência acidental de uma montagem errada de /proc.

Na pilha OCI, a especificação de runtime define o contrato declarativo do contêiner: processo, ambiente, rootfs, hooks, namespaces, cgroups e a lista de mounts, cuja ordem deve ser respeitada [17]. Já o *runc*, em sua especificação e em seu comportamento efetivo, mostra o lado operacional: procfs montado como filesystem padrão com `MS_NOEXEC`, `MS_NOSUID` e `MS_NODEV`; symlinks como `/dev/fd`, `/dev/stdin`, `/dev/stdout` e `/dev/stderr` criados depois; sincronização entre processo pai e processo init por pipe antes de o processo dentro do contêiner continuar [18]. Esse padrão é uma referência muito útil para uma implementação do zero, porque resolve exatamente os problemas que um design ingênuo costuma deixar de lado: ordem, sincronização, ownership do setup e limpeza.

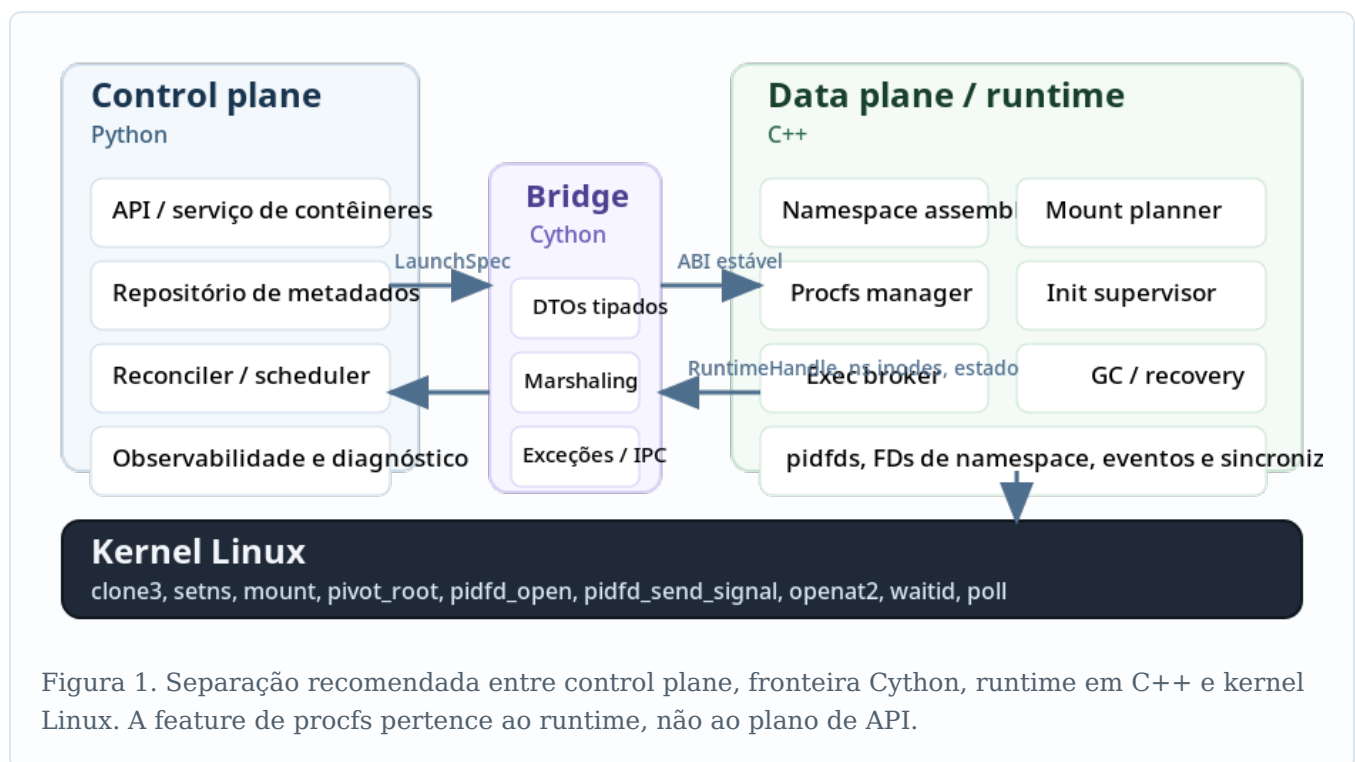
O *containerd*, por sua vez, deixa claro que não é ele próprio quem “entra no kernel e monta tudo”, mas que a função de lançamento e gerenciamento fino recai sobre runtimes e shims [16]. O shim é especialmente interessante como analogia para o projeto porque separa responsabilidades: ele faz a ponte entre control plane e processos do contêiner, atua na coleta de estado, espera saída do processo, e isola o gerenciador principal de detalhes como reaping e reconexão. Em um projeto Python + Cython + C++, isso sugere fortemente a adoção de um runtime/shim em C++ com ciclo de vida próprio, em vez de executar toda a mecânica de namespaces dentro do processo do interpretador.

No plano de segurança, a pilha OCI também ensina pela via negativa. A especificação e implementações como *runc* mantêm listas de *maskedPaths* e *readonlyPaths* para partes sensíveis de /proc, como `/proc/kcore`, `/proc/timer_stats`, `/proc/latency_stats`, `/proc/sched_debug`, `/proc/sys`, `/proc/sysrq-trigger` e outros caminhos [17] [18]. Mais importante ainda, as vulnerabilidades públicas do *runc* mostram que a existência da política não basta; a ordem de mount, o modo de mascaramento, a verificação de caminhos e a prevenção de corridas importam tanto quanto a intenção declarada [19] [20]. Para o projeto, isso implica que a montagem de /proc não pode ser tratada como uma operação solta. Ela precisa estar

subordinada a um *mount planner* com fases bem definidas e ownership exclusivo sobre a subárvore /proc.

Outro ponto relevante da documentação do Docker é a discussão de rootless e *usersns-remap*. A motivação é simples: mapear o “root” do contêiner para um range não privilegiado no host reduz a superfície de dano em caso de escape ou falha lógica [13] [14]. O projeto proposto deve absorver essa lição desde o início, mesmo que rootless chegue em uma fase posterior. Em termos práticos, isso quer dizer que o modelo de domínio e os contratos entre Python, Cython e C++ não podem assumir que toda operação sensível será sempre executada por um processo host-root; eles precisam acomodar a existência de mapas de UID/GID, sincronização para escrita em `uid_map` e `gid_map`, e feature probes de kernel.

Por fim, a documentação de métricas do Docker traz um lembrete operacional valioso: namespaces são materializados em `/proc/<pid>/ns`, e observadores que mantêm descritores desses arquivos indefinidamente podem, na prática, prender recursos de namespace além do necessário [15]. O manual geral de namespaces complementa essa visão ao afirmar que descritores abertos para entradas de `/proc/<pid>/ns` e até a existência de um `procfs` correspondente podem manter namespaces vivos [3]. Esse detalhe é muito fácil de ignorar quando o control plane é feito em uma linguagem de mais alto nível, mas ele é crucial para evitar “fantasmas” de runtime. Uma feature bem feita precisa ser correta não apenas no *create*, mas também no *delete* e na recuperação após falha.



A lição geral do Docker/OCI para este tema pode ser resumida assim: a semântica de /proc é decidida no runtime. O control plane declara intenção; o runtime materializa realidade. Esse é exatamente o encaixe desejável para o projeto proposto.

3. Arquitetura recomendada para Python, Cython e C++

O particionamento tecnológico sugerido pelo enunciado é sólido e, para essa feature, faz ainda mais sentido do que em um runtime genérico. A montagem de /proc, a criação de namespaces, o uso de pidfds, a sincronização pai-filho para mapas de usuário e a limpeza segura de mounts são operações fortemente acopladas ao kernel. Elas exigem controle fino sobre syscalls, descritores de arquivo, tratamento de sinais, isolamento de threads e rollback parcial em caso de falha. São, portanto, candidatas naturais ao C++. Em contrapartida, o catálogo de contêineres, os metadados, a API administrativa, a reconciliação de estado e a tradução das necessidades de produto para políticas de runtime são tarefas muito bem servidas por Python. O Cython aparece como camada de integração, desde que mantido fino e disciplinado.

3.1 O que deve ficar no control plane em Python

No control plane devem ficar os objetos de negócio e a orquestração do ciclo de vida: especificação do contêiner, políticas de isolamento, persistência de estado desejado, reconciliação de estado observado, emissão de eventos, autorização, exposição de API e diagnóstico de alto nível. O que importa aqui é que Python não deve carregar a responsabilidade de segurar processos filhos de contêiner por sinalização direta, esperar processos com semântica crítica de runtime ou manter descritores de namespace como parte do modelo central. O papel do control plane é declarar “o que deve existir” e registrar “o que de fato existe”, não executar o ritual sensível de criação do sandbox.

Do ponto de vista de modelagem, o control plane deveria persistir campos como *container_id*, *runtime_instance_id*, caminho do rootfs, política de procfs, perfil rootful/rootless, listas de paths mascarados e somente leitura, host PID do init para diagnóstico, inodes dos namespaces principais e timestamps de transição de estado. Já objetos como pidfd e FDs de namespace não pertencem ao banco de metadados porque são recursos de processo, não identidades de domínio. Se o runtime rodar em processo separado, esses handles permanecem sob custódia do runtime; o Python mantém apenas referências estáveis o bastante para reaquisição ou reconciliação.

3.2 O que deve ficar na camada Cython

A função ideal do Cython é ser uma fronteira de tipos e uma camada anticorrupção. Ele deve receber DTOs Python imutáveis ou pouco mutáveis, validá-los, traduzi-los para estruturas C++ com layout previsível e devolver respostas igualmente tipadas. Também é um bom lugar para converter erros de baixo nível em exceções mais expressivas, preservando errno e contexto técnico sem expor o código Python a uma avalanche de detalhes de ABI. O que o Cython não deve virar é um segundo runtime, repleto de lógica de namespace, chamadas de setns e estados parciais difíceis de depurar. Quando a cola acumula semântica demais, ela deixa de colar e começa a rachar.

Na prática, isso significa que a API Cython deveria operar sobre contratos como `LaunchSpec`, `ProcfsPolicy`, `ExecSpec` e `RuntimeHandle`, todos explícitos. O Python não deve enviar “dicionários livres” para o C++, assim como o C++ não deve devolver um punhado de inteiros sem nome esperando que o lado Python deduza o significado. Em uma feature tão sujeita a estados parcialmente válidos, nomes explícitos são parte da segurança.

3.3 O que deve ficar no runtime em C++

O runtime em C++ é o proprietário real da feature. É nele que devem morar serviços como *NamespaceAssembler*, *MountPlanner*, *ProcfsManager*, *InitSupervisor*, *ExecBroker* e *GarbageCollector*. É também nele que faz sentido encapsular `UniqueFd`, `PidFd`, `MountGuard`, `NamespaceFdSet` e demais wrappers RAI. Em um design maduro, esse runtime deve preferencialmente ser um processo separado do interpretador Python. Essa decisão tem três vantagens. Primeiro, reduz o raio de explosão de bugs de baixo nível. Segundo, evita interações delicadas entre threads do Python, GIL e restrições de `setns()` em processos multithread [5]. Terceiro, aproxima o sistema do desenho consolidado pela pilha Docker/containerd/runc, em que o plano de controle e o plano de execução não vivem no mesmo processo privilegiado [16] [18].

Esse runtime também é o lugar natural para manter *pidfds* vivos enquanto o contêiner existe. Um *pidfd* é preferível a um PID numérico porque resiste a reciclagem de identificadores, pode ser esperado via `waitid` ou `poll`, pode receber sinais de forma segura e pode servir de ponto de entrada para `setns` em fluxos de *exec* [10]. Em outras palavras, o *pidfd* é o tipo de referência operacional que o runtime deve custodiar, enquanto o control plane guarda metadados explicáveis e reconstituíveis.

3.4 Contrato entre as camadas

O contrato entre as camadas deve ser pequeno, estável e orientado a intenção. Um `LaunchSpec` precisa carregar `rootfs`, `argv` inicial, ambiente, diretório de trabalho, configuração de namespaces, política de `procfs`, listas de mounts reservados pelo runtime, lista de mounts do usuário fora das áreas reservadas, mapas de usuário e opções de `init` interno. Um `RuntimeHandle` precisa carregar, no mínimo, um identificador do runtime, o host PID do `init` para diagnóstico, os inodes de namespace observados na criação e o perfil de `procfs` efetivamente aplicado. Um `LaunchResult` também deveria conter um relatório de capabilities e fallbacks usados, para que o control plane saiba se foi usado `clone3`, se houve fallback para `clone`, se `subset=pid` estava disponível e assim por diante.

```
Python API  —> Cython DTO/ABI  —> Runtime C++
CreateContainer(spec)           —> LaunchSpec
ExecInContainer(handle, execspec) —> ExecSpec
DeleteContainer(handle)         —> TeardownRequest

Runtime C++ —> Cython —> Python
LaunchResult { runtime_id, init_host_pid, pid_ns_ino, mnt_ns_ino, user_ns_ino,
                proc_profile, warnings, timings }
ExecResult   { exec_id, started_at, host_pid, exit_code?, warnings }
Error        { category, errno_value, syscall_name, path?, namespace?, message }
```

Note que o contrato acima não expõe flags de mount ou ordens internas de syscalls ao Python. Isso é deliberado. O control plane lida com política e intenção; a tradução dessa política para a sequência operacional exata pertence ao runtime. Esse encapsulamento é importante para evitar que a API pública cristalize detalhes do kernel que, mais cedo ou mais tarde, precisarão mudar para incorporar correções de segurança ou novos recursos.

3.5 Metadados que valem a pena persistir

Como a feature lida com diagnósticos difíceis, alguns metadados específicos têm grande valor. Vale persistir, por exemplo, a string de namespace retornada por `readlink("/proc/<pid>/ns/<pid>")` e equivalentes para `mnt` e `user`, porque isso torna auditorias e testes muito mais simples [3] [15]. Vale também persistir a política de procfs aplicada, inclusive se houve uso de `subset=pid`, `hidepid` e grupo de observadores. Por outro lado, manter descritores abertos de namespace no control plane apenas “por conveniência” é um antipadrão, justamente porque eles podem prolongar a vida de namespaces que deveriam desaparecer [3] [15].

Essa distinção — persistir identidade observável, não reter handle operacional — ajuda a manter o sistema depurável sem introduzir vazamentos de recursos.

4. System Design da feature

A feature pode ser descrita como uma cadeia de transformações entre intenção declarativa e ambiente processual real. O usuário ou camada superior pede “suba um contêiner com isolamento de processos”. O sistema precisa converter isso em uma topologia de namespaces, uma árvore de mounts privada, um procfs correto, um processo PID 1 confiável e um conjunto de garantias de limpeza. Essa transformação só é segura quando o sistema trata /proc como parte do *sandbox assembly*, e não como um mount periférico.

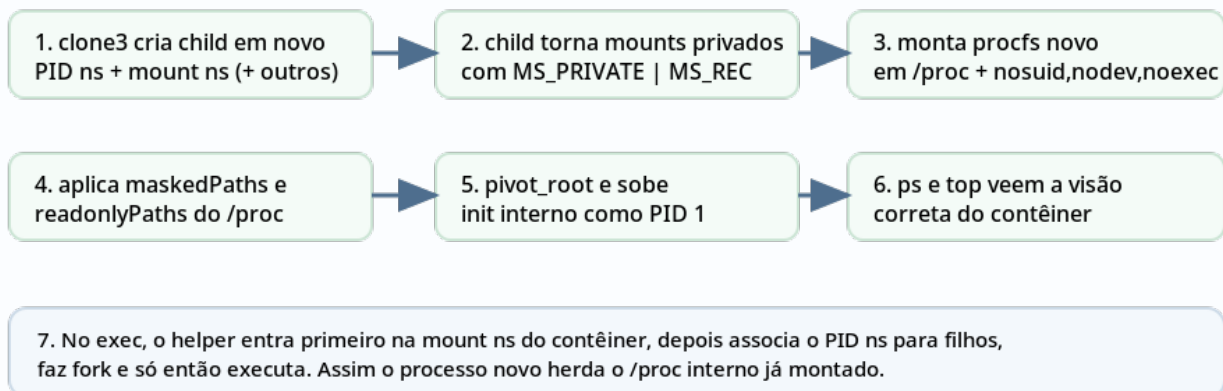
4.1 Fluxo de criação do contêiner

O fluxo recomendado começa no control plane. A API recebe a requisição, valida semântica de alto nível, persiste o estado desejado e compila um LaunchSpec imutável. Esse LaunchSpec é enviado ao runtime por meio do Cython ou, se o runtime estiver em processo separado, por uma camada de IPC. O runtime então executa um *feature probe* sobre o kernel: verifica disponibilidade de clone3, CLONE_PIDFD, pidfd_open, openat2, subset=pid e, quando aplicável, da nova API de mounts. Esse probe não é cosmético; ele define qual estratégia concreta será usada sem amarrar o sistema a checks frágeis por versão de kernel.

Depois da fase de capability detection, o runtime cria o processo-filho com o conjunto necessário de namespaces. Em um caso típico de contêiner isolado, isso inclui novo PID namespace e novo mount namespace, além de UTS, IPC, net e cgroup conforme a abrangência do projeto. Se houver rootless, user namespace entra junto. A partir desse ponto, o child executa a fase sensível: torna a árvore de mounts privada, prepara o rootfs, monta o procfs interno em /proc, aplica endurecimentos, realiza pivot_root e sobe o init interno, que por sua vez inicia o processo do usuário como filho direto ou supervisionado. O runtime pai só marca o contêiner como “running” após receber confirmação inequívoca de que o procfs já foi montado e o init interno está funcional.

Sequência correta

A visão de /proc é definida pela combinação entre PID namespace, mount namespace e o processo que faz a montagem.



Atalho incorreto

Entrar só no PID ns, ou bind-montar /proc do host, faz ps/top enxergarem a visão errada.

Figura 2. A sequência correta para materializar /proc depende da entrada prévia nos namespaces corretos e da montagem realizada pelo processo certo. O atalho “PID namespace sem mount namespace” é uma armadilha clássica.

Do ponto de vista de estados, é útil separar *Requested*, *Creating*, *Created*, *Running*, *Exited*, *Failed* e *Deleting*. A transição crítica é *Creating* → *Running*, que só deve ocorrer depois de /proc estar montado e verificado. Caso algo falhe entre o início de *Creating* e a subida do processo init, o sistema deve passar para *Failed* com tentativa ativa de rollback: desmontar o procfs se já houver sido montado, fechar descritores de namespace e registrar timings e causas técnicas.

4.2 Fluxo de exec dentro do contêiner

O fluxo de *exec* é tão importante quanto o de criação, porque é nele que muitos runtimes “quase corretos” revelam inconsistências. A entrada recomendada é um `RuntimeHandle` resolvido para o processo init do contêiner, de preferência por meio de `pidfd` [10]. O runtime deve abrir ou reutilizar as referências de namespace associadas ao init e criar um helper de *exec* dedicado, de preferência single-thread. Esse helper entra no user namespace, no mount namespace e nos demais namespaces que devem ser compartilhados com o contêiner. A seguir, faz `setns()` no PID namespace do alvo; como o manual explica, essa mudança afeta apenas os filhos futuros do helper, não o próprio helper [5]. O passo final é fazer `fork` e executar o processo desejado no filho.

O ponto central aqui é que o helper deve entrar no mount namespace do contêiner antes de criar o processo que executará o comando. Fazendo isso, o novo processo já nasce usando o mesmo /proc interno montado na criação do contêiner. Se o helper entrar apenas no PID namespace e permanecer com o mount namespace do host, o processo poderá nascer no PID

namespace “certo” para filhos, mas seguirá olhando para o /proc errado. É exatamente esse tipo de bug que leva operadores a desacreditar do runtime, porque o sistema parece parcialmente correto até o momento em que alguém roda ps ou top.

4.3 Perfis de procfs como política de produto

Uma forma elegante de conectar negócio e implementação é tratar a montagem de /proc como uma política explícita, e não como um booleano. O projeto deveria começar com pelo menos três perfis conceituais. O primeiro, *full_hardened*, monta um procfs completo, preservando arquivos globais e endurecendo caminhos sensíveis. Esse deve ser o padrão. O segundo, *tasks_only*, usa `subset=pid` quando o kernel suporta, buscando um ambiente mais enxuto para cenários que só exigem enumeração de processos. O terceiro, *shared_pid_debug*, existiria apenas se o produto decidir oferecer semântica semelhante a `--pid=host` ou compartilhamento de PID com outro contêiner; mesmo assim, a recomendação é montar um procfs novo no namespace compartilhado, e não bind-montar o procfs do host [1] [2] [12].

Tratar esses perfis como valor de domínio melhora a legibilidade do sistema. Em vez de uma miríade de flags dispersas, o operador e o código passam a falar na mesma linguagem: “este contêiner usa procfs completo endurecido”, “este contêiner usa tasks-only”, “este contêiner foi criado em modo de debug compartilhado”. Isso é DDD, mas também é manutenção operacional.

Dentro desses perfis, a decisão sobre `hidepid` merece tratamento separado. Em contêineres single-tenant, em que todos os processos pertencem ao mesmo domínio administrativo, o default mais pragmático costuma ser `hidepid=0`, preservando a experiência de ferramentas e depuração. Em ambientes com múltiplos usuários efetivos dentro do mesmo contêiner, ou quando se deseja limitar observabilidade transversal entre processos, `hidepid=2` com um grupo observador explícito pode ser uma opção melhor, explorando inclusive o parâmetro `gid=` do procfs [1]. O importante é não fixar essa decisão no código de mount; ela deve ser parte da política de procfs.

4.4 Governança da subárvore /proc

Uma decisão forte e recomendada é declarar a subárvore /proc como **área reservada do runtime**. Isso significa que mounts fornecidos pelo usuário não podem ter /proc nem descendentes como destino, salvo raríssimas exceções controladas pelo próprio runtime. Esse princípio evita que um caso de uso aparentemente inocente — por exemplo, bind-montar um arquivo em /proc/sysrq-trigger ou sobrepor parte de /proc — recrie classes inteiras de bugs e vulnerabilidades já conhecidas na indústria [19] [20].

Na prática, o *MountPlanner* deveria segmentar a árvore de montagem em fases. Primeiro entram os mounts estruturais pertencentes ao runtime, como o próprio /proc. Depois entram as operações de endurecimento da subárvore reservada. Só então mounts do usuário são considerados, desde que não colidam com áreas reservadas. Por fim entram symlinks e conveniências como /dev/fd e correlatos, quando aplicável [17] [18]. Esse planejamento em fases transforma ordem de operação em regra de produto, o que é muito mais seguro do que deixá-la implícita em código espalhado.

4.5 Observabilidade nativa da feature

Como se trata de uma feature de diagnóstico e de correção operacional, o sistema precisa ser observável por desenho. O runtime deve emitir eventos claros para “procfs montado”, “procfs endurecido”, “init iniciado”, “exec associado ao mount namespace correto” e “namespace coletado”. Também deve exportar métricas como tempo de montagem de /proc, falhas de setns, falhas de aplicação de *maskedPaths*, contêineres com namespaces pinados e tempo de limpeza no delete. No plano de API, vale a pena expor um endpoint de diagnóstico que devolva, para cada contêiner, os inodes de namespaces, a linha de mountinfo referente a /proc e o perfil efetivo de procfs. Isso reduz a dependência de inspeção manual por shell e simplifica muito a investigação de incidentes.

5. Low Level Design

Se o System Design define a topologia da solução, o Low Level Design define a ordem exata em que o runtime conversa com o kernel e como cada falha parcial será tratada. Nesta feature, ordem importa mais do que em grande parte do software de aplicação. Um único mount no momento errado, ou um único descritor de namespace mantido aberto além do necessário, basta para transformar uma implementação aparentemente funcional em uma fonte de inconsistência ou vazamento.

5.1 Estruturas e contratos de baixo nível

O contrato entre Python/Cython/C++ deve desembocar em estruturas pequenas, explícitas e fáceis de serializar. Uma estrutura de política para procfs, por exemplo, precisa capturar o perfil escolhido, a estratégia de hidepid, o grupo observador opcional e as listas finais de caminhos mascarados e somente leitura. Já o plano de lançamento precisa separar o que é mount estrutural, mount do usuário e hardening do runtime. Misturar tudo em uma lista única de mounts torna a ordenação implícita e, portanto, perigosa.

```
enum class ProcfsProfile {
    FullHardened,
    TasksOnly,
    SharedPidDebug
};

struct ProcfsPolicy {
    ProcfsProfile profile;
    int hidepid_mode;           // 0, 1 ou 2
    std::optional observer_gid;
    bool request_subset_pid;
    std::vector<std::string> masked_paths;
    std::vector<std::string> readonly_paths;
};

struct NamespaceOptions {
    bool new_user;
    bool new_pid;
    bool new_mount;
    bool new_uts;
    bool new_ipc;
    bool new_net;
    bool new_cgroup;
};

struct LaunchSpec {
    std::string rootfs;
    std::vector<std::string> argv;
    std::vector<std::string> env;
    std::string workdir;
    NamespaceOptions ns;
    ProcfsPolicy procfs;
    std::vector<MountSpec> runtime_mounts;
    std::vector<MountSpec> user_mounts;
    std::vector<IdMapEntry> uid_map;
```

```
std::vector<IdMapEntry> gid_map;
bool use_internal_init;
};
```

Acima, a decisão mais importante não está nos nomes, mas na separação de responsabilidades. O runtime monta e endurece a subárvore /proc; o usuário pode pedir mounts, mas não sobre áreas reservadas. Isso precisa aparecer na estrutura de dados, ou a regra acabará ficando só em comentários e validações tardias.

5.2 Pipeline rootful: sequência recomendada de syscalls

No caso rootful, o runtime deve preferir `clone3()` quando disponível, especialmente por permitir integração mais limpa com `CLONE_PIDFD` e por tornar o contrato de criação mais explícito [9] [10]. O pai cria pipes ou socketpairs de sincronização, prepara estruturas de resultado e invoca `clone3` com flags para os namespaces desejados. O filho resultante, que será o primeiro processo do novo PID namespace quando `CLONE_NEWPID` estiver presente, ainda não deve executar a carga do usuário. Antes disso ele precisa montar o ambiente.

A primeira ação relevante do child, já dentro do novo mount namespace, deve ser privatizar a árvore de mounts. Sem essa etapa, mounts subsequentes podem propagar indevidamente [4]. Depois disso, o runtime valida o rootfs. Essa validação deve ser defensiva: o caminho de /proc dentro do rootfs precisa ser um diretório legítimo e não pode ser um symlink hostil. Em kernels modernos, `openat2()` com resoluções restritas é a estratégia preferível para neutralizar escapes por symlink e ataques de caminho [23]. Em ambientes sem `openat2`, o fallback deve combinar `openat`, `fstatat`, inspeção de inodes e recusa explícita de componentes simbólicos.

Somente depois dessa validação o runtime monta um novo procfs no /proc do rootfs com flags de endurecimento mínimas. Na prática, isso significa algo como `nosuid,nodev,noexec`, com opções adicionais de `hidepid` e, se o perfil permitir, `subset=pid` [1] [18]. O detalhe crucial é que essa montagem deve acontecer quando o processo montador já está no PID namespace correto. Essa é a garantia semântica que faz ps e correlatos funcionarem da forma esperada [2].

Depois da montagem, o runtime aplica operações de endurecimento na subárvore /proc. O projeto deveria começar de um baseline inspirado na OCI e no *runc*, com caminhos como /proc/kcore, /proc/timer_stats, /proc/latency_stats e /proc/sched_debug mascarados, e porções como /proc/sys, /proc/irq, /proc/bus e correlatos somente leitura, ajustando a política conforme o produto [17] [18] [19] [20]. Em seguida, vem o restante da preparação do ambiente: eventualmente /dev, symlinks como /dev/fd, bind mounts necessários fora da área reservada, e então `pivot_root` ou fallback apropriado.

Encerrado o setup do filesystem, o runtime sobe o init interno. Esse init passa a ser o PID 1 do contêiner. Só então o payload do usuário deve ser iniciado. Se qualquer etapa falhar antes disso, o child deve reportar erro pelo pipe de sincronização e sair; o pai deve executar o rollback e nunca publicar o estado como *running*.

```

parent:
    validate(spec)
    make_sync_pipe()
    pidfd = clone3(CLONE_NEWPID | CLONE_NEWNS | ... | CLONE_PIDFD)
    wait_child_stage("ready-for-rootfs-or-error")

child:
    mount(NULL, "/", NULL, MS_REC | MS_PRIVATE, NULL)
    secure_resolve(rootfs, "/proc")
    mount("proc", rootfs + "/proc", "proc", MS_NOSUID | MS_NODEV | MS_NOEXEC, proc_opts)
    apply_proc_masks_and_readonly(rootfs, policy)
    setup_remaining_runtime_mounts(rootfs)
    pivot_root(...)
    exec_internal_init(...)

```

5.3 Pipeline rootless: sincronização pai-filho e mapas de identidade

No modo rootless, a sequência é parecida, mas há uma fase extra de sincronização para mapas de UID/GID. Quando o runtime cria o child com `CLONE_NEWUSER` junto dos demais namespaces, o child nasce no novo user namespace com capacidades ali, mas os mapas ainda precisam ser escritos pelo pai antes de certas operações seguirem [6]. O padrão recomendado, parecido com o usado historicamente pelo *runc*, é o child bloquear em um pipe; o pai escreve `/proc/<child>/setgroups` com `deny` quando necessário, depois escreve `uid_map` e `gid_map`, e só então libera o child para continuar [18].

Esse fluxo não deve ficar escondido em código genérico. É melhor assumi-lo como parte do desenho e até explicitá-lo nos contratos internos, porque ele altera materialmente o comportamento de erro, de timeout e de logging. Se a escrita de mapas falha, o erro não é “mount falhou”; é “precondição de identidade falhou”. Essa distinção é importante para operação e para testes.

Uma vez concluída essa sincronização, o child segue a mesma disciplina do modo rootful: privatiza mounts, valida rootfs, monta procfs interno e sobe o init. A diferença substancial é que certas operações e certos mounts podem depender das capacidades internas do user namespace ou de restrições adicionais do kernel [6] [14]. Portanto, o runtime precisa registrar claramente que backend foi usado, para que o plano de controle tenha diagnósticos precisos.

5.4 Implementação do init interno

O init interno não é um detalhe estético; ele é a âncora de sanidade do PID namespace. Como PID 1, ele deve esperar e reaproveitar todos os filhos órfãos ou terminados, repassar sinais relevantes ao processo principal do workload e consolidar um código de saída único para o contêiner [2]. Uma implementação simples e robusta em C++ pode ter um laço baseado em `waitid()` ou `waitpid()`, uma tabela mínima de processos monitorados e política explícita de forward de sinais. Em cenários mais elaborados, ele também pode registrar métricas de zombies reaped e tempo de shutdown.

Há uma razão adicional para essa camada no projeto: execs posteriores e processos auxiliares podem gerar órfãos dependendo do timing e das falhas. A documentação do *containerd*

destaca que, quando o shim não compartilha o PID namespace do contêiner, o init do próprio contêiner deve limpar órfãos adequadamente [16]. Ou seja, o init interno não é apenas uma questão de compatibilidade com sinais; ele é parte da estratégia de limpeza operacional.

5.5 O broker de exec e a ordem correta de `setns()`

O broker de exec deve operar em um processo helper dedicado, preferencialmente single-thread, para não colidir com restrições do `setns()` em processos multithread ou com estado compartilhado de filesystem [5]. O algoritmo recomendado é: abrir um `pidfd` do init ou reutilizar o já mantido pelo runtime; abrir referências dos namespaces desejados a partir do init; entrar primeiro no user namespace, quando aplicável; entrar depois no mount namespace; em seguida entrar em UTS, IPC, net, cgroup e demais namespaces compartilhados; por fim, fazer `setns()` no PID namespace, lembrando que esse passo apenas define o namespace dos filhos futuros [5] [10]. Depois disso, o helper faz `fork`, e o filho executa o comando.

Há dois motivos para essa ordem. O primeiro é técnico: certas trocas de namespace dependem de capacidades associadas ao user namespace. O segundo é semântico: entrar no mount namespace antes do `fork` garante que o processo resultante herdará o /proc correto do contêiner. Esse é o ponto onde a feature costuma ser perdida em implementações apressadas.

Outra recomendação forte é não deixar FDs de namespace abertos além do necessário. Eles são ferramentas de transição, não estado permanente. Após o `exec`, o helper deve fechá-los imediatamente, porque FDs abertos podem piná-los [3]. O mesmo vale para observadores e coletores de métricas: abrir, inspecionar, fechar.

5.6 Limpeza, coleta de lixo e namespaces pinados

A deleção correta de um contêiner não termina quando o processo principal sai. O runtime precisa sinalizar o init via `pidfd` ou mecanismo equivalente, aguardar o encerramento, desmontar o procfs interno e liberar quaisquer binds ou mounts auxiliares. Isso é importante não apenas por higiene, mas porque a própria existência de um procfs montado pode manter o PID namespace vivo [3]. Se o runtime matar o processo e esquecer o mount, terá criado um vazamento de namespace difícil de diagnosticar.

Em um sistema com control plane persistente, a recuperação após crash também precisa considerar esse fenômeno. Na retomada, o runtime deve varrer contêineres marcados como *running*, verificar se o host PID ainda existe, tentar reabrir `pidfd` quando possível [10], comparar inodes de namespace e verificar se mounts residuais continuam presentes. Esse reconciler de baixo nível é parte da feature, porque é ele quem evita contêineres zumbis ou namespaces órfãos após falhas do próprio runtime.

5.7 Endurecimento da subárvore /proc e prevenção de corridas de caminho

As vulnerabilidades recentes do *runc* merecem ser lidas como requisitos de projeto, não apenas como história alheia. Elas mostram que mascarar caminhos de /proc por bind mount de /dev/null ou aplicar `readonly/masked` em ordem inadequada pode abrir janelas de exposição inesperadas [19] [20]. O projeto precisa aprender com isso de duas maneiras. A primeira é

proibir que mounts do usuário incidam sobre a subárvore /proc. A segunda é tornar o código de resolução de caminhos robusto contra symlinks, renames e TOCTOU, idealmente com `openat2()`, e com verificação de inodes dos objetos usados como origem de bind mounts sensíveis [23].

Em kernels modernos, vale considerar o uso da API de mounts mais recente, com operações sobre árvores destacadas e movimentos atômicos, justamente para reduzir janelas de corrida. Em kernels mais antigos, o sistema deve cair para as operações clássicas, mas registrar o fallback e ativar testes de regressão específicos. O ponto não é perseguir novidade por si; é reduzir o espaço em que a implementação pode “achar que endureceu” algo que, sob concorrência hostil, ainda está exposto.

Caminho	Política inicial sugerida	Justificativa
/proc/kcore	Mascarado	É um dos caminhos clássicos de sensibilidade alta e aparece recorrentemente em políticas e advisories de runtime [17] [19].
/proc/timer_stats, /proc/latency_stats, /proc/sched_debug	Mascarados	São exemplos tradicionais de <i>maskedPaths</i> em perfis OCI e reduzem exposição de informações de kernel e scheduling [17].
/proc/keys e /proc/timer_list	Mascarados	Advisories do <i>runc</i> explicitam o risco associado a sua exposição indevida [19].
/proc/sys e /proc/sysrq-trigger	Somente leitura ou mascarado, conforme caminho	São áreas sensíveis para tunables globais do kernel; implementações vulneráveis já permitiram cópias graváveis desses caminhos [17] [20].
/proc/irq, /proc/bus, /proc/asound	Somente leitura	Compõem o baseline de endurecimento comum em perfis OCI [17].

5.8 Estratégia de fallback e feature probing

Compatibilidade não deve ser tratada como “se kernel $\geq$ X então faça Y”. Essa abordagem costuma envelhecer mal. O runtime deveria testar diretamente a disponibilidade e o comportamento de cada recurso relevante. Se `clone3` não estiver disponível, cai para `clone`. Se `CLONE_PIDFD` não estiver disponível na criação, tenta `pidfd_open()` em seguida [9] [10]. Se `subset=pid` não for aceito pelo procfs do kernel, o perfil *tasks_only* deve falhar com erro claro ou degradar de forma explicitamente sinalizada, nunca silenciosa [1]. Esse tipo de probe reduz dependência de adivinhações por versão e melhora a portabilidade entre distribuições e kernels corporativos.

6. Modelagem por Domain-Driven Design

Embora o tema pareça dominado por syscalls, a aplicação de Domain-Driven Design é útil justamente para impedir que o projeto vire um amontoado de detalhes de kernel sem linguagem comum. O problema de negócio aqui é oferecer um ambiente de execução isolado e coerente; namespaces, mounts e procfs são mecanismos, não a própria linguagem de produto. Um bom DDD ajuda a separar intenção de mecanismo e a impedir que o Python, o Cython e o C++ conversem em dialetos diferentes.

6.1 Bounded contexts

O primeiro *bounded context* é o de **Orquestração e Estado Desejado**. Nele vivem conceitos como contêiner, política de isolamento, perfil de procfs, imagem/rootfs, intenções de execução e transições de estado visíveis para API. É o contexto natural do Python. O segundo é o de **Execução de Runtime**. Nele vivem assembleia de namespaces, construção de mounts, init interno, exec broker, handles operacionais e rollback. É o contexto do C++. O terceiro é o de **Segurança e Isolamento**, transversal aos dois anteriores, mas com implementação majoritariamente no runtime. Ele contém decisões como áreas reservadas, mascaramento de caminhos, compatibilidade rootless e estratégias de fallback. O quarto é o de **Observabilidade e Diagnóstico**, que traduz inodes de namespace, mountinfo, eventos e métricas em algo explicável para o control plane e para o operador.

Esses contextos se comunicam, mas não devem se fundir. Quando o contexto de Orquestração começa a manipular descritores de namespace, ele está invadindo o contexto de Execução. Quando o runtime começa a “decidir” sozinho políticas de negócio sem receber uma abstração clara, ele está usurpando o contexto de Orquestração. A fronteira existe para conter acoplamento semântico.

6.2 Entidades, agregados e value objects

No contexto de Orquestração, o agregado principal é **Container**. Ele possui identidade própria, estado desejado, estado observado, referência ao rootfs, política de procfs, backend rootful/rootless e links para eventos e diagnósticos. Seu ciclo de vida atravessa a API e o repositório. No contexto de Runtime, um agregado natural é **TaskRuntime** ou **Sandbox-Runtime**, identificado por um *runtime_id*, que encapsula init, handles operacionais, namespace inodes observados e recursos a serem coletados na deleção.

Como *value objects*, vale modelar **ProcfsPolicy**, **NamespaceSet**, **MountPlan**, **UserMapping**, **RootfsDescriptor** e **ExecSpec**. Todos esses objetos são ideais para semântica de imutabilidade, comparação por valor e validação forte. **NamespaceSet**, por exemplo, pode conter os inodes de pid, mnt e user, além de referências efêmeras de runtime. **ProcfsPolicy** pode conter perfil, modo de hidepid, opções de subset e políticas de máscara. Modelá-los como objetos de valor reduz bugs de configuração e facilita testes.

Já **ExecSession** pode ser tratado como entidade do contexto de Runtime ou como objeto de processo transitório, dependendo do produto. Se a plataforma precisar auditar execs, rastrear duração, usuário efetivo e exit code de cada sessão, então *ExecSession* ganha identi-

dade e histórico. Caso contrário, pode permanecer um conceito efêmero emitido como evento.

6.3 Ubiquitous language

Sem uma linguagem ubíqua, essa feature vira um festival de mal-entendidos. O sistema deve distinguir explicitamente *host PID* e *namespace PID*. Deve distinguir *init interno* de *processo principal do workload*. Deve distinguir *procfs interno* de *bind mount de /proc do host*. Deve distinguir *perfil de procfs* de *mount options*. Deve distinguir *runtime handle* de *metadata handle*. Deve distinguir *área reservada* de *mount do usuário*. E deve distinguir *desired state* de *actual state*. Essas distinções parecem triviais até o dia em que um bug aparece porque um campo chamado apenas *pid* mudou de significado entre camadas.

Vale a pena documentar essa linguagem no próprio código e nas APIs. Se o Python recebe `init_host_pid` e `pid_namespace_inode`, o nome já carrega a pedagogia. Se recebe apenas `pid`, a ambiguidade vira dívida técnica.

6.4 Serviços de domínio e repositórios

Dentro do contexto de Orquestração, um **ContainerApplicationService** pode coordenar criação, deleção e exec, enquanto um **ContainerRepository** persiste o agregado. No contexto de Runtime, serviços como **NamespaceAssemblyService**, **ProcfsMountService**, **ExecIntoNamespaceService** e **RuntimeRecoveryService** encapsulam regras que não pertencem a uma única entidade. Isso é especialmente importante porque muita lógica relevante depende de combinação de objetos: *ProcfsPolicy* com *NamespaceOptions*, ou *TaskRuntime* com *ExecSpec*, por exemplo.

O repositório do control plane deve permanecer cego a detalhes de FD e syscalls. Já o “repositório” do runtime pode ser mais efêmero, potencialmente em memória mais journal local, porque lida com handles operacionais. O importante é não confundir persistência de domínio com retenção de recursos do kernel.

6.5 Eventos de domínio

Eventos ajudam muito nesse desenho. *ContainerRequested*, *ProcfsMounted*, *InitStarted*, *ExecStarted*, *ExecCompleted*, *ContainerExited*, *RuntimeRecovered* e *NamespaceGarbageCollected* são exemplos úteis. Eles permitem que o control plane reconcilie estados sem bloquear a API em toda a profundidade da operação, e também criam trilhas de auditoria importantes para incidentes. Em especial, ter um evento explícito de *ProcfsMounted* ajuda a materializar a ideia de que a feature não é implícita no “container running”; ela é um marco operacional verificável.

6.6 Cython como camada anticorrupção

Em DDD, a camada anticorrupção existe para impedir que um modelo conceitual seja contaminado por outro. Neste projeto, o Cython desempenha exatamente esse papel. O mundo Python pensa em contêineres, políticas e estados. O mundo C++ pensa em FDs, flags, pipes e rollback. O Cython deve traduzir um no outro sem permitir que erro, códigos crus de sys-

call, suposições de layout de memória e detalhes de mount API vazem para o modelo de domínio do control plane. Ao mesmo tempo, deve impedir que abstrações vagas demais cheguem ao runtime, porque “faça aí a coisa certa com proc” não é um contrato executável.

Essa visão reduz bastante o risco de a camada de cola virar uma mistura confusa de Python procedural e C++ parcialmente embrulhado. A cola deve proteger a linguagem do domínio, não erodi-la.

7. Padrões de projeto aplicáveis

Nesta feature, padrões de projeto são úteis quando traduzem variabilidade real do problema em estruturas explícitas. Eles não devem ser usados para enfeitar o código; devem ser usados para reduzir acoplamento, estabilizar contratos e manter a ordem de operações sob controle. Alguns padrões GoF se encaixam muito bem, e alguns padrões não-GoF, como RAIL, são praticamente obrigatórios.

7.1 Strategy para perfis de procfs

Strategy é o padrão mais natural para representar perfis como *full_hardened*, *tasks_only* e *shared_pid_debug*. Cada estratégia sabe compor opções de mount, validar compatibilidade com o kernel, ajustar listas padrão de caminhos mascarados e readonly e emitir diagnósticos específicos. O benefício não é apenas elegância. Ele impede que dezenas de if/else sobre perfil fiquem espalhados por montagem, testes e endpoints de observabilidade. A política de procfs passa a ser um objeto executável e testável.

7.2 Builder para *LaunchPlan* e *MountPlan*

Builder é apropriado porque a criação do ambiente exige várias decisões encadeadas e uma ordem canônica. Um *LaunchPlanBuilder* pode começar da intenção declarada, aplicar defaults, validar impossibilidades, decidir o backend rootful/rootless e então gerar um *MountPlan* com fases explícitas. Sem Builder, é comum que cada parte do código “acrescente mais um mount” em uma lista compartilhada, e a ordem final passe a depender de acaso e histórico de manutenção.

7.3 Abstract Factory para backends rootful e rootless

Abstract Factory ajuda a encapsular a criação de serviços de baixo nível diferentes para ambientes rootful e rootless. Ambos expõem a mesma interface conceitual — criar sandbox, entrar em namespaces, montar procfs, iniciar init —, mas divergem na maneira como obtêm capacidades e escrevem mapas de usuário. Essa separação é melhor do que saturar o runtime com flags contextuais que atravessam todos os métodos.

7.4 Command para operações de ciclo de vida

Command é adequado para operações como *CreateContainer*, *StartContainer*, *ExecProcess*, *DeleteContainer* e *GarbageCollectSandbox*. Cada comando encapsula parâmetros, validações, métricas, logging e rollback. Isso combina especialmente bem com um control plane que deseja enfileirar, reexecutar, registrar ou auditar operações. Em vez de uma API interna monolítica, o sistema passa a ter ações nomeadas e serializáveis.

7.5 State para o ciclo de vida do contêiner

State é útil para garantir que transições como *Creating* → *Running* e *Running* → *Deleting* tenham regras explícitas. A vantagem prática aqui é impedir que operações sejam chamadas

em estados incoerentes. Um *exec*, por exemplo, só faz sentido em *Running*. Um rollback de mounts só faz sentido enquanto o sandbox ainda está em *Creating* ou *Failed*. Colocar essas regras em um State object ou em uma máquina de estados formal melhora robustez e testabilidade.

7.6 Facade para o runtime a partir do Python

Facade é o padrão certo para o lado Python enxergar o runtime. O control plane não deveria conhecer classes como `NamespaceFdSet`, `MountGuard` ou detalhes de pipes de sincronização. Ele deveria falar com algo como `RuntimeFacade.create()`, `RuntimeFacade.exec()` e `RuntimeFacade.delete()`, recebendo resultados de alto nível. Isso simplifica a API, reduz vazamento de abstrações e torna a camada Cython mais estável.

7.7 Adapter para a fronteira Cython/C++

Adapter aparece naturalmente na tradução entre DTOs Python e estruturas C++. Um adaptador bem desenhado converte strings, listas e enums de forma segura, traduz exceções e normaliza códigos de erro. O valor do padrão aqui é disciplinar a fronteira. Em vez de chamadas ad hoc com parâmetros posicionais difíceis de evoluir, a integração se torna uma adaptação explícita entre dois mundos.

7.8 Observer para eventos de runtime

Observer é apropriado para o fluxo de eventos de runtime. O `init` pode emitir saída; o `pidfd` pode sinalizar término; o runtime pode observar falhas de montagem; o control plane pode assinar eventos de vida útil. Esse padrão é especialmente útil se o sistema pretende oferecer API assíncrona, streaming de logs ou reconciliação desacoplada do caminho síncrono da requisição.

7.9 RAII e Scope Guard como padrões essenciais de sistema

Embora não pertençam ao catálogo GoF, **RAII** e **scope guards** são indispensáveis em um runtime C++ que manipula FDs, mounts e recursos de kernel. Um `UniqueFd` que fecha automaticamente no destrutor, um `MountGuard` que desmonta no rollback, um `ChildSyncGuard` que fecha pipes corretamente e um `PidfdHandle` que centraliza operações de sinalização/espera reduzem a classe inteira de bugs de vazamento e cleanup parcial. Em software de baixo nível, esses padrões fazem mais diferença do que metade da literatura de OO clássica.

Tomados em conjunto, esses padrões organizam o sistema de modo coerente: `Strategy` define política, `Builder` define plano, `Abstract Factory` seleciona backend, `Command` encapsula ações, `State` protege transições, `Facade` estabiliza a API, `Adapter` protege a fronteira entre linguagens e `RAII` garante que o kernel não fique com resíduos quando algo falha no meio do caminho.

8. Requisitos funcionais e não funcionais

Em uma feature como esta, requisitos bem escritos evitam dois problemas comuns: a falsa sensação de que “o contêiner sobe, então está tudo certo”, e a tendência de transformar decisões críticas em defaults implícitos e difíceis de verificar. Os requisitos abaixo foram formulados para cobrir correção semântica, segurança, observabilidade, capacidade de operação rootless, estratégia de *exec* e limpeza de recursos. Eles não devem ser lidos como mera checklist burocrática; são, na prática, a definição de pronto da feature.

8.1 Requisitos funcionais sugeridos

ID	Requisito	Critério de aceitação sugerido
RF-01	Ao criar um contêiner com PID namespace privado, o runtime deve montar um novo procfs em /proc a partir de um processo que já esteja no PID namespace alvo e dentro de um mount namespace isolado.	Em teste de integração, <code>ps -o pid,ppid,comm</code> executado dentro do contêiner mostra apenas processos do namespace, e o <code>mountinfo</code> contém uma entrada <code>proc /proc proc</code> pertencente ao mount namespace do contêiner.
RF-02	O sistema deve oferecer um perfil padrão <i>full_hardened</i> para /proc, com flags mínimas de endurecimento e política padrão de paths mascarados/somente leitura.	A criação do contêiner registra o perfil efetivo e aplica, no mínimo, <code>nosuid</code> , <code>nodev</code> e <code>noexec</code> ; caminhos sensíveis configurados aparecem endurecidos em inspeção pós-criação.
RF-03	O sistema deve oferecer um perfil opcional <i>tasks_only</i> que utilize <code>subset=pid</code> quando o kernel suportar a opção.	Quando o perfil é solicitado em kernel compatível, o mount de /proc inclui <code>subset=pid</code> ; em kernel incompatível, o runtime falha com diagnóstico explícito ou executa fallback declarado ao operador.
RF-04	O caminho /proc e sua subárvore devem ser considerados área reservada do runtime.	Mounts solicitados pelo usuário para /proc ou descendentes são rejeitados na validação de spec com erro específico de <i>ReservedMountViolation</i> .
RF-05	A implementação deve oferecer um init interno como PID 1 do contêiner quando o modo de supervisão padrão estiver ativo.	Em teste com processos órfãos e zombies, o contêiner não acumula zombies persistentes; o PID 1 repassa sinais de término ao workload e produz código de saída consolidado.
RF-06	A operação de <i>exec</i> deve entrar no mount namespace do contêiner e só depois associar o PID namespace para filhos, realizando fork antes do <i>exec</i> do comando.	Um <code>ps</code> disparado por <i>exec</i> enxerga a mesma visão processual e o mesmo /proc do contêiner original; um teste de regressão confirma que a implementação não depende de entrar apenas no PID namespace.
RF-07	O runtime deve manter referência operacional estável ao init por <code>pidfd</code> sempre que o kernel permitir.	A camada de runtime consegue sinalizar e aguardar a saída do init sem depender apenas de PID numérico; eventos de encerramento são produzidos por espera do <code>pidfd</code> ou por fallback claramente registrado.
RF-08	O sistema deve suportar modo rootless quando o kernel permitir user namespaces e mounts correspondentes.	Em ambiente de teste rootless, o runtime cria user namespace, escreve <code>uid_map</code> e <code>gid_map</code> de forma sincronizada e consegue montar o procfs interno dentro do sandbox.
RF-09		

ID	Requisito	Critério de aceitação sugerido
	A API de diagnóstico deve expor o perfil efetivo de procfs, os inodes de namespaces principais e metadados de montagem do contêiner.	Uma chamada de inspeção devolve, no mínimo, <code>pid_ns</code> , <code>mnt_ns</code> , <code>user_ns</code> , perfil de procfs, host PID do init e informação resumida de mountinfo para /proc.
RF-10	A deleção do contêiner deve desmontar o procfs interno e liberar referências de namespace mantidas pelo runtime.	Após <i>delete</i> , testes de varredura não encontram procfs residual do contêiner nem namespace pinado por FD ou mount deixado pelo runtime.

8.2 Requisitos não funcionais sugeridos

Os requisitos não funcionais abaixo são particularmente importantes porque a feature opera em uma superfície em que erros podem parecer pequenos no código, mas enormes em produção. Uma montagem errada de /proc não é apenas um bug visual; ela altera diagnóstico, isolamento, segurança e comportamento de ferramentas. Por isso, atributos de qualidade como corretude, segurança, recuperabilidade e observabilidade precisam ser requisitos explícitos.

ID	Categoria	Requisito	Critério de aceitação sugerido
RNF-01	Corretude semântica	A visão de /proc deve ser consistente com o PID namespace do contêiner em criação inicial, <i>exec</i> , reinicialização do runtime e exclusão.	Todos os cenários de integração preservam coerência de ps, /proc/1/status e mountinfo.
RNF-02	Segurança	A implementação deve minimizar exposição de caminhos sensíveis de /proc, bloquear mounts sobre áreas reservadas e usar resolução de caminho resistente a symlink/TOCTOU.	Testes de segurança e regressões inspiradas em advisories do <i>runc</i> não conseguem sobrepor /proc nem burlar paths mascarados.
RNF-03	Compatibilidade baseada em feature probe	Decisões de runtime devem usar detecção de capacidades do kernel, não apenas versão nominal.	Logs e diagnóstico registrados se foram usados clone3, pivot_root, subset=pid e demais fallbacks.
RNF-04	Observabilidade	A feature deve emitir eventos, métricas e diagnósticos suficientes para depuração sem shell interativo no host.	Existem métricas e eventos para montagem de /proc, falhas de setns, tempos de cleanup e leaks detectados.
RNF-05	Recuperabilidade	O sistema deve tolerar crash do control plane e do runtime, oferecendo mecanismo de reconciliação e garbage collection.	Após reinício do runtime, o reconciler detecta contêineres vivos, reabre handles quando possível e limpa resíduos de mounts mortos.
RNF-06	Isolamento de falhas	Erros do runtime de baixo nível não devem derrubar o processo principal de API nem corromper o repositório de metadados.	O runtime roda isolado do interpretador Python ou em domínio de falha equivalente; falhas geram estados consistentes e recuperáveis.
RNF-07	Testabilidade	O design deve permitir testes unitários de planejamento e testes de integração privilegiados de criação, exec e cleanup.	Builders, estratégias e validadores são testáveis isoladamente; cenários de namespace/mount possuem suíte de integração automatizada.
RNF-08	Manutenibilidade	As responsabilidades de política, montagem e domínio devem permanecer separadas entre Python, Cython e C++.	Não há syscalls de namespace no control plane; não há decisão de negócio espalhada no runtime; a camada Cython permanece fina.

ID	Categoria	Requisito	Critério de aceitação sugerido
RNF-09	Desempenho observável	A criação do contêiner deve medir explicitamente o custo das etapas de procfs e namespace setup para evitar regressões invisíveis.	Timings por fase são persistidos ou exportados; regressões de latência podem ser detectadas em CI e produção.
RNF-10	Portabilidade operacional dentro do alvo Linux	A feature deve funcionar de forma previsível em diferentes distribuições Linux do escopo suportado.	Diferenças de kernel e distribuição são acomodadas por probes, fallbacks e matriz de compatibilidade documentada.

Vale observar que vários requisitos se reforçam mutuamente. Reservar /proc como subárvore do runtime melhora segurança, mas também simplifica manutenção. Usar pidfd melhora correte operacional, mas também recuperabilidade. Persistir namespace inodes melhora observabilidade, mas também facilita testes. Essa interdependência é um bom sinal: indica que o desenho está atacando causas estruturais, não apenas sintomas.

9. Plano de implementação por fases

Como a feature toca kernel, isolamento, segurança e experiência operacional, a implementação não deve ser feita em um único salto. Um plano por fases reduz risco, preserva capacidade de diagnóstico e permite validar hipóteses antes de adicionar complexidade. A progressão proposta abaixo segue a dependência real entre as capacidades: primeiro a semântica mínima correta, depois a ergonomia operacional, depois o endurecimento e, por fim, recuperação e maturidade de produção.

9.1 Fase 0 — Infraestrutura de teste, probes e linguagem ubíqua

Antes de montar /proc em produção, o projeto deve preparar a infraestrutura que tornará a feature testável. Isso inclui um harness de integração privilegiado para Linux, probes para clone3, pidfd, openat2 e subset=pid, definição explícita de DTOs como LaunchSpec e ProcfsPolicy, e documentação da linguagem ubíqua do domínio. O objetivo dessa fase é evitar que a equipe entre diretamente em syscalls sem um vocabulário comum e sem feedback automatizado.

9.2 Fase 1 — MVP rootful com PID namespace privado e procfs interno correto

A primeira fase funcional deve atacar a essência do problema e nada além dela. O runtime cria novo PID namespace e novo mount namespace, privatiza a árvore de mounts, monta um procfs novo em /proc com nosuid,nodev,noexec, sobe um init interno simples e executa o payload. O critério de saída dessa fase não é “o processo inicia”; é “ps dentro do contêiner reflete o namespace correto e /proc/1 é o init interno”. Nessa fase, ainda é aceitável deixar rootless e parte do endurecimento mais fino para depois, desde que a correção semântica esteja consolidada.

9.3 Fase 2 — exec consistente, pidfd e init interno robusto

Com a criação básica estável, a prioridade seguinte é o caminho de exec. O runtime passa a manter pidfd do init, introduz um broker de exec dedicado e implementa a ordem correta de setns(). Em paralelo, o init interno deixa de ser apenas um wrapper mínimo e passa a reaper sinais, gerenciar shutdown e consolidar exit codes. O objetivo dessa fase é garantir que a experiência operacional seja consistente também após a criação do contêiner, especialmente para debug, shells interativos e rotinas administrativas.

9.4 Fase 3 — Endurecimento de /proc e proteção da subárvore reservada

Nessa fase entram a lista padrão de *maskedPaths* e *readonlyPaths*, a recusa de mounts do usuário sobre /proc, a resolução segura de caminhos com openat2() ou fallback endurecido e testes de regressão inspirados nas vulnerabilidades do *runc* [19] [20] [23]. Também é o momento certo para introduzir o perfil *tasks_only* e formalizar a política padrão *full_hardened*. O critério de saída não é apenas passar em testes felizes, mas demonstrar que o runtime resiste às classes de erro mais conhecidas da área.

9.5 Fase 4 — Rootless e user namespaces

Com o caminho rootful estabilizado, entra o backend rootless. Essa fase adiciona user namespace, sincronização para uid_map/gid_map, tratamento de setgroups e adequação das capacidades do runtime [6] [13] [14] [18]. O objetivo é reproduzir a mesma semântica de procfs interno correto em um contexto sem privilégios totais no host. Aqui, o valor do feature probe se torna evidente, porque nem todo ambiente Linux aceitará os mesmos caminhos de montagem.

9.6 Fase 5 — Recuperação após crash, garbage collection e maturidade de produção

Na fase final entram os mecanismos de reconciliação após reinício do runtime, varredura de namespaces pinados, coleta de mounts órfãos, telemetria completa e documentação operacional. O sistema passa a tratar o runtime não como processo infalível, mas como componente recuperável. Essa fase também deve incluir benchmarks de latência por etapa, testes de estresse com criação/deleção em massa e auditoria dos eventos de diagnóstico exportados para o control plane.

Sequência recomendada de entrega

Se for preciso escolher entre “mais features” e “mais correção”, escolha correção. Um contêiner que sobe rápido mas mostra o /proc errado é pior do que um contêiner com menos features que sobe com semântica certa. A feature em pauta é infraestrutura de confiança; sem ela, o restante do runtime perde credibilidade operacional.

10. Estratégia de testes e validação

A validação dessa feature precisa combinar testes unitários, integração privilegiada, regressões de segurança e inspeção observável do estado de kernel. Testes exclusivamente unitários não capturam a semântica de namespaces; testes exclusivamente manuais não escalam nem preservam regressão. O sistema deve, portanto, nascer com uma pirâmide de testes específica para runtime.

10.1 Testes unitários

Em nível unitário, o foco deve recair sobre tudo o que é determinístico sem depender do kernel real: composição de *ProcfsPolicy*, construção de *MountPlan*, validação de áreas reservadas, normalização de caminhos, tradução Python-Cython-C++, máquina de estados do contêiner e mapeamento de erros. É perfeitamente possível testar, por exemplo, que um spec contendo mount em /proc/sys é rejeitado antes de qualquer syscall, ou que o perfil *tasks_only* nunca é escolhido silenciosamente quando o feature probe informa ausência de subset=pid.

10.2 Testes de integração privilegiados

Os testes de integração são o coração da validação. Eles devem criar contêineres reais em ambiente Linux dedicado ou CI com privilégios suficientes, executar comandos dentro deles e inspecionar o resultado. O conjunto mínimo inclui: verificar que `readlink /proc/self/ns/pid` dentro do contêiner corresponde ao inode esperado; verificar que `awk '/^NSpid:/{print}' /proc/1/status` contém PID 1 como último elemento; verificar que `ps -o pid,ppid,comm` mostra apenas processos do contêiner; verificar que `grep " /proc " /proc/self/mountinfo` aponta para um mount de proc com as flags certas; e verificar que `top -b -n1` funciona no perfil *full_hardened*.

Também é importante ter um teste de regressão negativa que demonstre o erro clássico. O harness deve criar deliberadamente um helper que entra apenas no PID namespace, sem entrar no mount namespace, e provar que `ps` ou a inspeção de /proc ficam incorretos. Esse tipo de teste é valioso porque preserva o conhecimento arquitetural, impedindo que uma futura “otimização” destrua a semântica correta por engano.

10.3 Testes específicos por perfil de procfs

Para o perfil *full_hardened*, a suíte deve validar compatibilidade com `ps` e `top`, além da presença das proteções esperadas em caminhos sensíveis. Para o perfil *tasks_only*, a suíte deve validar que a enumeração por processo funciona, mas também registrar explicitamente a degradação esperada de ferramentas dependentes de arquivos globais. Isso é importante para que a equipe e os usuários entendam que se trata de uma escolha de produto, não de um bug intermitente.

10.4 Testes de segurança e regressão de advisories

As advisories públicas do *runc* devem inspirar uma suíte de regressão permanente [19] [20]. Ela deve cobrir tentativas de sobrepor a subárvore /proc com mounts do usuário, tentativas de explorar symlinks no rootfs, tentativas de acessar caminhos mascarados após reorderings de mount e tentativas de obter cópias graváveis de caminhos que deveriam estar somente leitura. Mesmo que a implementação não replique a arquitetura exata do *runc*, as classes de ataque são relevantes e devem ser reencenadas.

10.5 Testes de cleanup e namespaces pinados

Uma classe de testes frequentemente negligenciada é a de limpeza. O sistema deve criar contêiner, encerrá-lo, deletá-lo e então verificar se o procfs foi desmontado e se o namespace não ficou preso por FDs remanescentes. Também vale simular o crash do runtime entre “processo saiu” e “cleanup terminou”, reiniciar o runtime e validar que o reconciler descobre e remove resíduos corretamente. Esses testes são os que diferenciam um protótipo funcional de um componente que pode operar continuamente.

Cenário	Verificação principal	Resultado esperado
Criação básica com <i>full_hardened</i>	ps, /proc/1/status, mountinfo de /proc	Visão de processos correta, PID 1 interno, mount de procfs presente com endurecimento mínimo.
exec no contêiner	Comparar inodes de namespace e saída de ps entre contêiner e sessão de exec	A sessão de exec herda a visão correta do mesmo /proc interno.
Perfil <i>tasks_only</i>	Validar opção subset=pid e comportamento de ps/top	ps segue funcional; degradação de ferramentas dependentes de arquivos globais é explícita e documentada.
Tentativa de mount do usuário sobre /proc	Submeter spec inválido e observar validação	Requisição rejeitada antes da criação do sandbox.
Delete e coleta de lixo	Encerrar contêiner, reiniciar runtime, varrer mounts e inodes	Sem procfs residual, sem namespace pinado pelo runtime.

Em resumo, a estratégia de testes precisa provar três coisas ao mesmo tempo: que a feature funciona quando tudo dá certo, que falha da maneira certa quando o kernel ou o spec não permitem a operação, e que não deixa sujeira quando algo falha no meio do caminho.

11. Conclusão

A montagem de /proc com visão correta do PID namespace é uma daquelas features que parecem pequenas quando vistas pela API e enormes quando vistas pelo kernel. No papel, a demanda parece resumir-se a “fazer o ps funcionar dentro do contêiner”. Na prática, ela exige decisões cuidadosas de arquitetura, modelagem, sincronização, segurança e cleanup. O núcleo da solução, entretanto, é simples e deve permanecer simples na mente da equipe: o contêiner precisa de um procfs novo, montado no mount namespace correto por um processo que já esteja no PID namespace correto. Sem isso, tudo o mais vira remendo.

Quando essa tese é levada a sério, as demais decisões se alinham. O control plane em Python mantém estado desejado, metadados e políticas. O Cython traduz contratos entre camadas e protege a linguagem do domínio. O runtime em C++ executa syscalls, gerencia pidfds, faz rollback e cuida do init interno. O DDD organiza a linguagem ubíqua. Os padrões de projeto contêm a variabilidade. Os requisitos tornam o comportamento verificável. E o plano por fases reduz risco sem sacrificar a direção correta.

Do ponto de vista de produto, a recomendação final é inequívoca: trate /proc como infraestrutura de runtime de primeira classe, reserve sua subárvore ao runtime, faça do perfil *full_hardened* o padrão, mantenha *tasks_only* como opção avançada, implemente *exec* com entrada prévia no mount namespace e use init interno com limpeza rigorosa. Esse conjunto de decisões aproxima o projeto da maturidade observada na pilha Docker/containerd/runc sem copiar cegamente sua complexidade. Copia-se o princípio certo: o controle alto fica em cima; a realidade do kernel fica embaixo.

Referências

1. **[1]** `proc(5)` — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man5/proc.5.html>.
2. **[2]** `pid_namespaces(7)` — Linux manual page. Disponível em: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html.
3. **[3]** `namespaces(7)` — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man7/namespaces.7.html>.
4. **[4]** `mount_namespaces(7)` — Linux manual page. Disponível em: https://man7.org/linux/man-pages/man7/mount_namespaces.7.html.
5. **[5]** `setns(2)` — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man2/setns.2.html>.
6. **[6]** `user_namespaces(7)` — Linux manual page. Disponível em: https://man7.org/linux/man-pages/man7/user_namespaces.7.html.
7. **[7]** `ps(1)` — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man1/ps.1.html>.
8. **[8]** `top(1)` — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man1/top.1.html>.
9. **[9]** `clone(2)` / `clone3` — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man2/clone.2.html>.
10. **[10]** `pidfd_open(2)` — Linux manual page. Disponível em: https://man7.org/linux/man-pages/man2/pidfd_open.2.html.
11. **[11]** Linux kernel documentation — /proc filesystem. Disponível em: <https://docs.kernel.org/filesystems/proc.html>.
12. **[12]** Docker CLI reference — docker container run. Disponível em: <https://docs.docker.com/reference/cli/docker/container/run/>.
13. **[13]** Docker Engine security — usersns-remap. Disponível em: <https://docs.docker.com/engine/security/usersns-remap/>.
14. **[14]** Docker Engine security — rootless troubleshoot. Disponível em: <https://docs.docker.com/engine/security/rootless/troubleshoot/>.
15. **[15]** Docker Engine — Runtime metrics and namespaces. Disponível em: <https://docs.docker.com/engine/containers/runmetrics/>.
16. **[16]** containerd Runtime v2 README. Disponível em: <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md>.

17. **[17]** OCI Runtime Specification — config.md. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/main/config.md>.
18. **[18]** runc libcontainer SPEC.md. Disponível em: <https://github.com/opencontainers/runc/blob/main/libcontainer/SPEC.md>.
19. **[19]** runc advisory GHSA-9493-h29p-rfm2. Disponível em: <https://github.com/opencontainers/runc/security/advisories/GHSA-9493-h29p-rfm2>.
20. **[20]** runc advisory GHSA-qw9x-cqr3-wc7r. Disponível em: <https://github.com/opencontainers/runc/security/advisories/GHSA-qw9x-cqr3-wc7r>.
21. **[21]** util-linux issue #660 — nsenter and ps view. Disponível em: <https://github.com/util-linux/util-linux/issues/660>.
22. **[22]** procps-ng top source tree. Disponível em: <https://gitlab.com/procps-ng/procps/-/blob/master/src/top/top.c>.
23. **[23]** openat2(2) — Linux manual page. Disponível em: <https://man7.org/linux/man-pages/man2/openat2.2.html>.